WEEK 1 · LECTURE 1

Large Language Models and Query Engineering

What LLMs are, where they came from, and what we will build with them

BSc Data Science and Artificial Intelligence · Al-Zaytoonah University of Jordan · Dr. Akram Fatayer · 2026/2027

TODAY

By the end of this lecture you can...

01

State what a language model computes, in one sentence

02

Trace the path from n-gram models to transformer LLMs

03

Name today's main model families, open and closed

04

List what LLMs do reliably and where they fail

A language model assigns a probability to the next token

```
P( next token | all previous tokens )
```

Sample a token from this distribution, append it, and repeat: that loop is all text generation is.

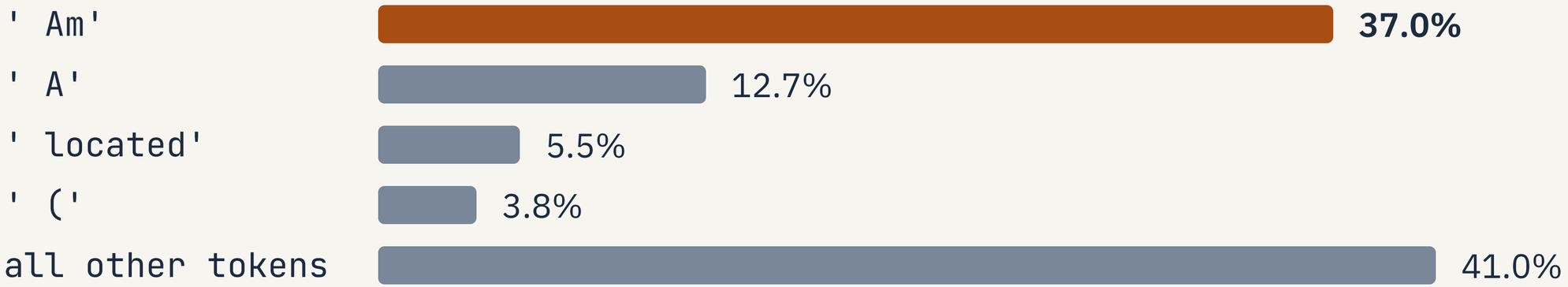
A **large** language model computes this distribution with a transformer network of billions of parameters, trained on trillions of tokens of text.

AN EXAMPLE

The model sees a prefix and scores every possible continuation

"The capital of Jordan is ___"

«عاصمة الأردن هي ___»



FROM N-GRAMS TO TRANSFORMERS

Eight decades of predicting the next word

1948

Shannon models English as a statistical source of letters and words

2003

Bengio et al.: a neural network language model that learns word embeddings

2013

word2vec makes dense word embeddings cheap to train

2014–15

Sequence-to-sequence RNNs and attention transform machine translation

2017

The Transformer: attention only, no recurrence, fully parallel training

2018

GPT and BERT: pre-train once on raw text, then adapt to tasks

2020

GPT-3 shows few-shot learning from examples in the prompt

2022

ChatGPT: instruction tuning and RLHF bring LLMs to the public

2023–26

Open-weight models, reasoning models, tool use and agents

An n-gram model counts which words follow which

The Markov assumption

Only the last n – 1 words matter. A trigram model looks back two words.

```
P(w | "capital of") =
count("capital of w")
÷ count("capital of")
```

Counted from a corpus

"capital of" followed by…	Count
Jordan	412
France	388
the	1,205
Tafilah	0
All words, in total	20,000

Counts are illustrative

WHY COUNTING HIT A WALL

Three problems, one fix: learn representations instead of counting

Sparsity

Most word sequences never appear in any corpus, so their count is zero.

No similarity

"Amman" and "Irbid" are unrelated symbols; nothing learned about one transfers to the other.

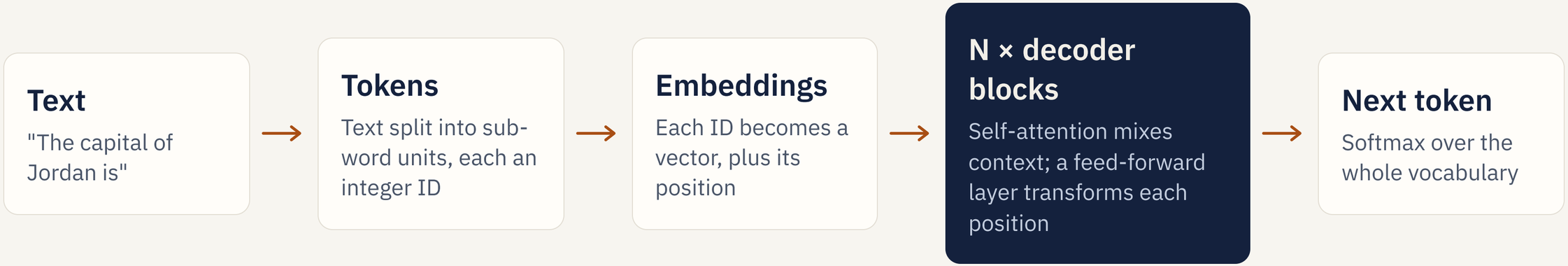
Short memory

Tables grow exponentially with n, so context stays a few words long.

Neural language models map each word to a dense vector (an embedding). Similar words get similar vectors, so knowledge generalises, and the network, not a table, decides how much context to use.

INSIDE AN LLM, AT A GLANCE

Text goes in as tokens and comes out as a probability distribution



> **Generation loop:** pick a token, append it to the input, run the whole network again. One forward pass produces one token.

WHAT MAKES IT "LARGE"

Same objective, a hundred times more parameters in one year

1.5B

GPT-2 parameters, 2019

175B

GPT-3 parameters, 2020

?

Today's frontier models: sizes are no longer published

Capability grew with three ingredients scaled together: **parameters**, **training data** and **compute**. Scaling laws describe how loss falls as each grows.

FROM TEXT PREDICTOR TO ASSISTANT

A chat assistant is a base model trained in two more stages

STAGE 1

Pre-training

Next-token prediction on trillions of tokens of web, books and code.

Result: a base model that continues text but does not follow instructions

→

STAGE 2

Instruction tuning

Fine-tuning on curated pairs of instructions and good responses.

Result: answers questions and follows a requested format

→

STAGE 3

Preference alignment

Learning from ranked responses, with RLHF or DPO.

Result: more helpful, honest and harmless replies

Stage 1 uses most of the compute; stages 2 and 3 shape the behaviour. Reasoning models add a further stage: reinforcement learning on problems with checkable answers (week 7).

THE LANDSCAPE, OCTOBER 2026

Who builds today's LLMs

Closed-weight frontier

Used through an API or app

- Anthropic: Claude
- OpenAI: GPT
- Google: Gemini
- xAI: Grok

Open-weight

Weights downloadable, run anywhere

- Meta: Llama 4
- Alibaba: Qwen 3.5
- DeepSeek: V4
- Google: Gemma 4
- Moonshot: Kimi K3

Arabic-focused

Built in the region, for Arabic

- Jais (MBZUAI/Core42, UAE)
- Falcon-H1 Arabic (TII, UAE)
- ALLaM (HUMAIN/SDAIA, KSA)
- Fanar (Qatar)

AN ENGINEERING CHOICE

Open or closed weights is a trade-off, not a ranking

	Closed-weight API	Open-weight, self-hosted
Capability	Usually the strongest	Close behind, at lower cost
Data privacy	Prompts leave your institution	Data can stay on your servers
Cost	Pay per token	Pay for GPUs and operations
Customisation	Prompting; limited fine-tuning	Full fine-tuning, LoRA
Transparency	Weights and data hidden	Weights inspectable; data often not
Licence	Provider's terms of service	Varies: MIT, Apache 2.0, custom

WHAT LLMS DO WELL

LLMs are strongest when the answer is in the text you give them

Transform

Summarise, translate, rewrite for a new audience or tone

Extract

Pull names, dates and fields from documents into structured data

Classify

Label sentiment, topic or intent with few or no examples

Answer from sources

Answer questions grounded in supplied documents

Write code

Generate, explain and debug programs and queries

Draft

Produce first versions of emails, reports and plans for a human to check

WHERE LLMS FAIL

Every failure traces back to how the model works

Failure	Example	Root cause
Hallucination	Invents a plausible paper citation	Optimised for plausible text, not truth
Stale knowledge	Unaware of events after training	Fixed training cut-off
Letter counting	Miscounts letters in a word (small models)	Sees tokens, not characters
Prompt sensitivity	Rephrasing changes the answer	Output depends on every input token
Prompt injection	Obeys instructions hidden in a web page	Instructions and data share one input
Uneven languages	Weaker on Arabic dialects than English	Less training data per language

WHAT "QUERY ENGINEERING" MEANS

We engineer everything between the question and a trustworthy answer

Prompt
Instructions, examples, output format
Weeks 6–8

→

Retrieval
Find the right documents to put in context
Weeks 9–10

→

LLM
Chosen and understood, not treated as magic
Weeks 1–5

→

Tools
Search, code, databases called by an agent
Week 11

→

Evaluation
Measure quality, safety and cost
Weeks 12–13

You will rarely train an LLM. You will often build the system around one, and that is where most of the quality comes from.

COURSE ROADMAP

Three parts, then two weeks for your project

Weeks 1–5

How LLMs work

- Tokens and embeddings
- Attention and transformers
- Pre-training and decoding
- Fine-tuning and alignment

Weeks 6–8

Prompt engineering

- Zero- and few-shot prompts
- Chain-of-thought, decomposition
- Structured outputs
- Project proposal in week 7

Weeks 9–13

Building systems

- Semantic search and RAG
- Agents and text-to-SQL
- Evaluation
- Safety and security

Weeks 14–15: project studio and demos · **Week 16:** final exam · **Every week:** one concept session, one lab session

ASSESSMENT

Coursework 60%, final exam 40%, no midterm

Component	Weight	When	What
Assignments	30%	Every lab; six mini-assignments	Lab submissions and individual take-home extensions, each graded out of 10
Team project	30%	Phases weeks 7, 10, 12, 14; demo week 15	Build and evaluate an app with retrieval and tools, teams of 3–4
Final exam	40%	Week 16	Written, comprehensive

USING AI IN THIS COURSE

Use AI tools openly, and own every line you submit

Allowed, with disclosure

Assignments and the project: brainstorming, debugging, editing. State which tool you used and for what.

Not allowed

Exams. Pasting personal or confidential data into third-party AI services.

You may be asked to explain

Any submission can be followed by a short oral check. Work you cannot explain is not graded as yours.

Why this policy

In a course about LLMs, using them well and critically is part of what you learn.

WHAT COMES NEXT

Next week: how text becomes tokens and vectors

This week's lab

In the GPU lab: set up your environment, run your first open-weight model, and try to break it

Read

Alammar & Grootendorst, *Hands-On Large Language Models*, chapter 1

Think about

Does the same sentence cost the same number of tokens in Arabic and in English?

Exit question: in one sentence, what does a language model compute?